FoundationPose vs. EKF-Based Stereo Pose Tracking for Manipulator Visual Servoing

Akanksha Singal and Sidd Vohra
Carnegie Mellon University
{asingal2, svohra}@andrew.cmu.edu

***Abstract*— Robust visual servoing is critical for autonomous manipulation in unstructured environments such as warehouses and homes. Visual servoing requires accurate estimation of the relative pose between the robot end-effector and the target grasp point. We compare two approaches to pose estimation in an iterative visual servoing loop: a learning-based perception front-end (DINOv2 + SAM2 + Depth Anything V2) that produces per-frame observations, and an Extended Kalman Filter (EKF) that fuses those observations into a temporally filtered 3D position estimate. We integrate both into a unified pipeline on an xArm manipulator with a ZED Mini camera, comparing an EKF-based position-based visual servoing (PBVS) controller against a baseline image-based visual servoing (IBVS) system. We describe our system architecture, the EKF formulation, and report on implementation progress toward a quantitative evaluation of closed-loop servoing performance.**

I. INTRODUCTION

Visual servoing uses visual feedback to guide a robot toward a desired configuration. In applications like warehouse pick-and-place and household manipulation, the robot must localize a target object and iteratively correct its end-effector pose until it reaches a grasp point. The quality of this process depends directly on how well the perception system estimates the object's pose at each control iteration. Errors in pose estimation do not simply add noise to the trajectory; they compound over time through the control loop, and can cause the robot to oscillate, overshoot, or diverge entirely from the desired grasp point.

There are two classical formulations for visual servoing [3]. Image-based visual servoing (IBVS) works entirely in pixel coordinates, mapping the error between current and desired image features to robot velocities through an image Jacobian. Because it avoids explicit 3D reconstruction, IBVS is simple and robust to calibration errors. However, IBVS can produce unintuitive Cartesian trajectories, struggles with large displacements, and has no way to reason about depth or distance to the target. Position-based visual servoing (PBVS) reconstructs the 3D pose of the target in the camera frame and computes a Cartesian velocity command that drives the object to a desired position. PBVS gives straight-line trajectories in Cartesian space, but its performance is tied directly to the accuracy of the underlying 3D pose estimate.

Recent foundation models for perception open up new possibilities for building PBVS systems that work on novel objects without task-specific training. DINOv2 [5] provides dense, semantically rich patch-level features from a self-supervised Vision Transformer that transfer well across visual domains. SAM2 [6] produces high-quality segmentation masks from minimal prompts (a bounding box or a few points) and supports temporal mask propagation across video frames via logit passing. Depth Anything V2 [7] estimates monocular depth from a single RGB image, providing relative depth that can be calibrated to metric scale. Together, these three models can localize, segment, and estimate the depth of an arbitrary object given only a single reference photo.

However, foundation model outputs are inherently per-frame: each image is processed independently with no temporal context. In a servoing loop where the robot is actively moving and the scene is changing, this independence means that frame-to-frame jitter in the detection centroid or depth estimate feeds directly into the control commands. A noisy centroid at one frame causes a jerky velocity command, which moves the robot, which changes the image, which produces another noisy centroid. In practice this can produce oscillatory behavior near the goal or slow convergence.

The Extended Kalman Filter (EKF) [2] provides a principled framework for smoothing noisy per-frame observations over time. By maintaining a probabilistic state estimate (position and velocity) along with a covariance matrix that tracks uncertainty, the EKF can dampen measurement noise, propagate predictions forward during brief tracking failures, and provide the controller with uncertainty-aware estimates. The EKF also provides a natural mechanism for incorporating the robot's own motion (egomotion compensation), since the camera is mounted on the moving end-effector.

In this project, we build a complete visual servoing system that combines foundation model perception with EKF-based state estimation, and compare it against a direct IBVS baseline that uses raw per-frame detections. Both systems run on real hardware (xArm manipulator, ZED Mini camera) with per-frame metrics logging for quantitative evaluation. Our goal is to characterize the practical trade-offs: does the EKF's temporal filtering and uncertainty tracking provide meaningful improvements in convergence speed, trajectory smoothness, and final accuracy compared to feeding raw per-frame observations directly to a controller?

II. RELATED WORK

Visual servoing. Chaumette and Hutchinson [3] provide a comprehensive treatment of visual servo control, covering

both IBVS and PBVS formulations, their stability properties, and the interaction matrix (image Jacobian) that relates image feature velocities to camera velocities. Their framework underpins most modern visual servoing systems. A key insight from their analysis is that IBVS is locally stable around the desired configuration but can fail for large displacements, while PBVS provides global convergence guarantees but is sensitive to pose estimation errors.

6-DoF pose estimation. FoundationPose [1] achieves state-of-the-art 6-DoF pose estimation and tracking from RGB-D input paired with either a CAD model or a small set of reference images. It has been benchmarked extensively on the BOP and YCB-Video datasets. However, it operates per-frame without temporal filtering, and its reliance on a known 3D model or RGB-D input limits applicability in settings with novel objects or monocular cameras.

EKF for visual tracking. The use of Kalman filtering for visual tracking has a long history [2]. The standard approach is to maintain a state vector (typically position and velocity in 3D) and update it with measurements derived from image features. The EKF linearizes the nonlinear measurement model (pinhole projection) around the current estimate at each step. Tao et al. [4] recently proposed a perception-control coupled approach for visual servoing of textureless objects using a keypoint-based EKF. Their work is close in spirit to ours, but they use learned keypoint detectors as the measurement source, whereas we use foundation model features (DINOv2 + SAM2).

Foundation models for robotics perception. DINOv2 [5] has been shown to produce patch-level features that transfer across visual domains for matching, retrieval, and segmentation tasks. SAM2 [6] extends the Segment Anything model with video-level mask propagation, enabling real-time tracking by passing low-resolution logits between consecutive frames. Depth Anything V2 [7] provides monocular depth estimation that, while producing relative rather than metric depth, can be converted to approximate metric scale through a simple affine calibration. These models have individually been applied to robotic tasks, but to our knowledge, no prior work has combined all three into an integrated perception front-end feeding an EKF for closed-loop visual servoing on real hardware.

III. THEORETICAL BACKGROUND

A. Pinhole Camera Model

A point $\mathbf{p} = [X, Y, Z]^T$ in the camera frame projects to pixel coordinates (u, v) via:

$$u = f_x \frac{X}{Z} + c_x, \qquad v = f_y \frac{Y}{Z} + c_y \tag{1}$$

where f_x, f_y are the focal lengths in pixels and (c_x, c_y) is the principal point. The inverse operation (backprojection) given a known depth Z recovers the 3D point: $X = (u - c_x)Z/f_x$, $Y = (v - c_y)Z/f_y$.

B. Extended Kalman Filter

We use an EKF to track the target object's 3D position and velocity in the camera frame. The state vector is:

$$\mathbf{x} = [X,\, Y,\, Z,\, \dot{X},\, \dot{Y},\, \dot{Z}]^T \in \mathbb{R}^6 \tag{2}$$

where (X, Y, Z) is the object position in metres and $(\dot{X}, \dot{Y}, \dot{Z})$ is velocity in m/s, both in the camera coordinate frame.

Process model. We assume constant-velocity dynamics with egomotion compensation. Given time step Δt and the robot end-effector velocity $\mathbf{v}_{\text{robot}}$ expressed in the camera frame:

$$\mathbf{x}_{k|k-1} = \mathbf{F}\,\mathbf{x}_{k-1} - \begin{bmatrix} \mathbf{v}_{\text{robot}}\,\Delta t \\ \mathbf{0}_3 \end{bmatrix} + \mathbf{w}_k \tag{3}$$

where $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}\Delta t)$ is process noise scaled by the time step, and the state transition matrix is:

$$\mathbf{F} = \begin{bmatrix} \mathbf{I}_3 & \Delta t\,\mathbf{I}_3 \\ \mathbf{0}_3 & \mathbf{I}_3 \end{bmatrix} \in \mathbb{R}^{6\times 6} \tag{4}$$

The egomotion subtraction accounts for the fact that when the camera (mounted on the end-effector) moves in direction $+X$, the object appears to shift in direction $-X$ in the camera frame. Without this compensation, the EKF would interpret robot-induced image motion as object motion and produce incorrect velocity estimates. The covariance prediction follows the standard form: $\mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1}\mathbf{F}^T + \mathbf{Q}\Delta t$.

Measurement model. The measurement vector combines the 2D pixel centroid of the detected object with a metric depth estimate:

$$\mathbf{z} = \begin{bmatrix} u \\ v \\ Z \end{bmatrix}, \qquad h(\mathbf{x}) = \begin{bmatrix} f_x X/Z + c_x \\ f_y Y/Z + c_y \\ Z \end{bmatrix} \tag{5}$$

This is the standard pinhole projection (Eq. 1) augmented with a direct depth observation. The measurement Jacobian $\mathbf{H} = \partial h/\partial \mathbf{x}$ evaluated at the current state estimate is:

$$\mathbf{H} = \begin{bmatrix} f_x/Z & 0 & -f_x X/Z^2 & 0 & 0 & 0 \\ 0 & f_y/Z & -f_y Y/Z^2 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \tag{6}$$

The velocity states $(\dot{X}, \dot{Y}, \dot{Z})$ do not appear directly in the measurement model, so the rightmost three columns of $\mathbf{H}$ are zero. The velocity is estimated indirectly: the process model couples position and velocity, and the position corrections from each measurement update propagate through the covariance to constrain the velocity as well.

Update step. Given a measurement $\mathbf{z}_k$ with noise covariance $\mathbf{R} = \text{diag}(\sigma_u^2, \sigma_v^2, \sigma_Z^2)$, we compute:

$$\mathbf{y}_k = \mathbf{z}_k - h(\mathbf{x}_{k|k-1}) \qquad \text{(innovation)} \tag{7}$$

$$\mathbf{S}_k = \mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^T + \mathbf{R} \qquad \text{(innovation covariance)} \tag{8}$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}^T\mathbf{S}_k^{-1} \qquad \text{(Kalman gain)} \tag{9}$$

$$\mathbf{x}_k = \mathbf{x}_{k|k-1} + \mathbf{K}_k\mathbf{y}_k \qquad \text{(state update)} \tag{10}$$

For the covariance update, we use the Joseph form rather than the standard $\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k\mathbf{H})\mathbf{P}_{k|k-1}$, because the Joseph

Fig. 1. Baseline IBVS pipeline. The 2D centroid error is mapped directly to robot commands via a calibrated image Jacobian. No temporal filtering or 3D reconstruction is performed.

form is algebraically equivalent but numerically more stable when the Kalman gain is large:

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k\mathbf{H})\,\mathbf{P}_{k|k-1}\,(\mathbf{I} - \mathbf{K}_k\mathbf{H})^T + \mathbf{K}_k\,\mathbf{R}\,\mathbf{K}_k^T \tag{11}$$

2D-only update. Running the depth model on every frame is expensive. On frames where we skip depth estimation, we perform a partial update using only the pixel centroid (u, v). This uses the top two rows of $\mathbf{H}$ (a 2×6 matrix) and the corresponding 2×2 block of $\mathbf{R}$. The 2D-only update still constrains the lateral position (X, Y) while allowing the depth Z to evolve under the process model.

C. Control Laws

IBVS baseline. The pixel error $\mathbf{e} = [e_x, e_y]^T$ between the detected object centroid and the image center is mapped to robot end-effector displacements through a 2×2 image Jacobian $\mathbf{J}$ (with units px/mm):

$$\begin{bmatrix} \Delta y \\ \Delta z \end{bmatrix}_{\text{robot}} = -\alpha\,\mathbf{J}^{-1}\,\mathbf{e} \tag{12}$$

where α is a proportional gain. We estimate $\mathbf{J}$ through an online optical-flow calibration procedure: the robot makes small known displacements in Y and Z, and we measure the resulting pixel shifts using Lucas-Kanade feature tracking, giving two columns of $\mathbf{J}$. A constant forward approach velocity is added along the robot X axis.

PBVS (EKF-based). The EKF-filtered 3D position $\mathbf{t}_{\text{obj}}$ is compared against a desired position $\mathbf{t}^* = [0, 0, Z^*]^T$ (centered on the optical axis at a target depth Z^*):

$$\mathbf{v}_{\text{cam}} = -\lambda\,(\mathbf{t}_{\text{obj}} - \mathbf{t}^*) \tag{13}$$

where λ is a proportional gain. The velocity is clamped per-axis for safety, then transformed from the camera frame to the robot base frame via the camera-to-robot rotation $\mathbf{R}_{\text{cam}\to\text{robot}}$ before being sent to the robot.

IV. System Architecture and Methodology

Our system consists of three modules: a perception front-end, a state estimator, and a servo controller. We implement two complete pipelines that share the same perception module but differ in how the observations are processed and fed to the robot. Fig. 1 shows the baseline IBVS pipeline and Fig. 2 shows the EKF-PBVS pipeline.

A. Perception: Detect-Then-Track

We adopt a two-phase strategy. Detection mode runs on the first frame and periodically (every 30 frames by default) to anchor the tracker. Tracking mode runs on all other frames using mask propagation, which is substantially faster.

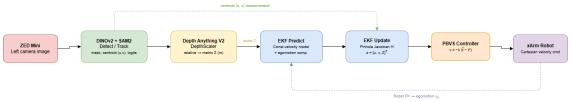

Fig. 2. EKF-PBVS pipeline. The perception front-end produces a 2D centroid and metric depth, which the EKF fuses into a filtered 3D position. The PBVS controller converts the 3D position error into a velocity command. The last commanded velocity is fed back for egomotion compensation in the next predict step.

Detection. Given a reference image of the target object (an RGBA image where the alpha channel separates foreground from background), we extract dense patch-level features from both the reference and the live scene using DINOv2 ViT-B/14. The image is divided into 14×14 pixel patches, and DINOv2 produces a 768-dimensional feature vector for each patch. For every scene patch, we compute its mean cosine similarity to all foreground reference patches, producing a spatial similarity heatmap. This heatmap is thresholded at a configurable percentile (93rd by default), morphologically cleaned, and the connected component with the highest mean similarity is selected to produce a bounding box. We also compute a complementary feature similarity map using a truncated ResNet-18 backbone (output stride 32, 512-d features), and fuse the two maps via weighted average to improve robustness. The bounding box is then used as a prompt for SAM2, which returns a binary segmentation mask and low-resolution logits $\ell \in \mathbb{R}^{1\times 256\times 256}$.

Tracking. On subsequent frames, we bypass DINOv2 entirely and instead feed the previous frame's SAM2 logits as a `mask_input` prompt on the new image. We also supply point prompts: the previous centroid as a positive point, and a set of background points (mined from the intersection of recent mask histories) as negatives. This gives SAM2 enough context to propagate the mask without re-running any feature extraction, achieving 5–10$\times$ faster per-frame processing. We monitor tracking quality by computing the IoU between the current and previous masks. If IoU drops below 0.35, we interpret this as drift and fall back to detection mode. Additionally, the first good detection is saved as a stable "anchor" mask and logits, which serve as a fallback if the tracker loses the object entirely.

Centroid extraction. From the binary mask, we compute a robust centroid using morphological closing (to bridge gaps caused by printed textures on box surfaces), extraction of the largest external contour, and the center of its minimum-area bounding rectangle. This procedure is more stable than a naive moments-based centroid for rectangular objects with internal lines or text.

B. Depth Estimation

We use Depth Anything V2 with a ViT-S encoder to produce a relative depth map from each RGB frame. Since the output is relative rather than metric, we convert it via an affine model: $Z_{\text{metric}} = s \cdot d_{\text{relative}} + b$, where s and b are calibration parameters. The object's metric depth is estimated as the median relative depth under the segmentation mask, converted to metres.

To reduce computational cost, we run depth inference only on every 5th frame with a valid detection (or on the very first detection, to initialize the EKF). On intermediate frames, the EKF propagates the depth forward using its constant-velocity process model, while the 2D-only measurement update continues to constrain the lateral position.

C. EKF Integration

At each frame, the processing proceeds as follows. (1) **Predict:** propagate the state and covariance forward using the constant-velocity model (Eq. 3), compensating for egomotion using the velocity from the last servo command. (2) **Update:** if a depth measurement is available and exceeds 5 cm (to reject invalid near-zero readings), perform the full 3-DOF update. Otherwise, perform the 2D-only update. (3) **Output:** the filtered position $[X,Y,Z]^T$ is passed to the PBVS controller (Eq. 13), and the scalar uncertainty $\sqrt{\text{tr}(\mathbf{P}_{1:3,1:3})}$ is displayed in the visualization overlay.

The EKF is initialized from the first valid (u,v,Z) measurement by backprojecting the centroid to 3D and setting the initial velocity to zero with a broad covariance. If the depth model is unavailable at startup, the EKF seeds itself with the PBVS target depth as a prior so the filter can begin operating. If the EKF has not yet initialized (no valid measurement has arrived), the system automatically falls back to the IBVS baseline.

D. Metrics Logging

We instrument both pipelines with a CSV logger that records, for every frame: wall-clock timestamp, frame index, time since last frame, detection mode (detect vs. track), raw pixel centroid, EKF-filtered position and velocity, position uncertainty, metric depth, the servo command sent to the robot (dx, dy, dz in mm), the Euclidean position error, and the total per-iteration wall-clock time. This allows us to compute all evaluation metrics in post-processing without modifying the control loop itself.

E. Hardware Setup

Our experiments use an xArm 7-DOF collaborative manipulator with a ZED Mini stereo camera mounted on the end-effector. The camera provides a 1280×720 left image at 30 fps. We use only the left camera image (not stereo depth) so that the pipeline generalizes to monocular setups. DINOv2, SAM2, and Depth Anything V2 run on a single GPU. The EKF and servo controller run on CPU. Communication with the xArm uses the xArm Python SDK over Ethernet.

Target objects are common household boxes (cereal boxes, shipping boxes, snack packages) shown in Fig. 3. No CAD models or prior 3D knowledge of the objects is required: the only input is a single reference photo with a transparent background.

V. PROGRESS AND REMAINING WORK

A. Completed

Some initial result videos for real world experiments can be found here.

- DINOv2 + SAM2 detect-then-track perception pipeline with anchor-based drift recovery, negative point mining, and robust centroid extraction.
- IBVS baseline controller with online optical-flow Jacobian calibration.
- EKF pose tracker with constant-velocity process model, pinhole projection measurement model, analytical Jacobian, Joseph-form covariance update, egomotion compensation from servo commands, and 2D-only partial update fallback.
- Full integration of the EKF into the live servo loop, with automatic fallback to IBVS when the filter has not yet initialized.
- PBVS controller with proportional gain, dead zone, and per-axis velocity clamping.
- Depth Anything V2 integration with affine depth scaling and every-5th-frame scheduling.
- Per-frame CSV metrics logger recording all quantities needed for post-hoc evaluation.
- Complete real-hardware setup (xArm + ZED Mini) tested end-to-end in both IBVS and EKF-PBVS modes.

B. Remaining Work

- Integrate FoundationPose as an additional perception back-end, feeding its 6-DoF pose estimates into the same EKF and PBVS controller.
- Run closed-loop servoing experiments comparing all three configurations (IBVS baseline, EKF + DINOv2/SAM2, EKF + FoundationPose) across multiple target objects and starting configurations.
- Collect quantitative metrics from the logged CSV data: final position error (cm), number of iterations to convergence within a 1 cm tolerance, path efficiency (actual trajectory length divided by straight-line distance), and per-iteration runtime broken down by perception, EKF, and servo stages.
- Perform EKF noise covariance sensitivity analysis by sweeping the process noise $\mathbf{Q}$ and measurement noise $\mathbf{R}$ parameters and measuring their effect on convergence speed and steady-state accuracy.
- Evaluate robustness under degraded conditions: partial occlusion of the target object and varied ambient lighting.
- Final analysis, comparison, and writeup.

C. Scope Changes

Our original proposal focused on comparing FoundationPose against an EKF-based tracker. We began implementation with a DINOv2 + SAM2 perception front-end rather than FoundationPose for two practical reasons: FoundationPose requires a known CAD model and RGB-D input, while DINOv2 + SAM2 works from a single reference photo with monocular RGB, making it easier to get an end-to-end pipeline running on real hardware first. We still plan to integrate FoundationPose as an additional perception back-end in the coming weeks, so that the final comparison will include three configurations: raw IBVS (baseline), EKF-filtered

Fig. 3. Target objects used in our experiments. These are common household boxes with varying textures, colors, and sizes. No CAD models are needed; the only input per object is one reference photo.

DINOv2+SAM2 (PBVS), and EKF-filtered FoundationPose (PBVS). This staged approach lets us isolate the contribution of the EKF independently from the choice of perception model.

D. Updated Timeline

Weeks 1–2 (Done)	DINOv2 + SAM2 perception pipeline, IBVS baseline, initial EKF implementation.
Weeks 3–4 (Done)	EKF integration into servo loop, PBVS controller, depth estimation, metrics logging, real hardware testing.
Week 5	Integrate FoundationPose, run comparison experiments, collect metrics, Q/R sweeps.
Week 6	Analysis, final writeup, presentation.

References

[1] B. Wen, W. Yang, J. Kautz, and S. Birchfield, "FoundationPose: Unified 6D Pose Estimation and Tracking of Novel Objects," in *Proc. IEEE/CVF CVPR*, 2024.

[2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[3] F. Chaumette and S. Hutchinson, "Visual Servo Control, Part I: Basic Approaches," *IEEE Robotics and Automation Magazine*, vol. 13, no. 4, pp. 82–90, 2006.

[4] A. Tao, J. Yang, S. Oparnica, and W. Xue, "Perception-Control Coupled Visual Servoing for Textureless Objects Using Keypoint-Based EKF," arXiv preprint arXiv:2602.06834, 2026.

[5] M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features without Supervision," *Trans. on Machine Learning Research*, 2024.

[6] N. Ravi, V. Gabeur, Y.-T. Hu, et al., "SAM 2: Segment Anything in Images and Videos," in *Proc. International Conference on Learning Representations (ICLR)*, 2025.

[7] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.